TECHNICAL UNIVERSITY OF KOŠICE
FACULTY OF ELECTRICAL ENGINEERING AND INFORMATICS

Problem Set 6

Bomberman Game

2023

Mykyta Lukianytsia, D6

Acknowledgement

My gratitude to Ryan Reich from https://stackoverflow.com/questions/2509679/how-to-generate-a-random-integer-number-from-within-a-range stackoverflow post for a great random numbers generation algorithm.

Table of Contents

Introduction. Short description of task and documentation 4

1. How to compile and run the program 5

1.1. Debian-based systems 5

1.2. Arch-based systems 5

2. Solutions I used to write the game. 7

2.1. Game in general 7

2.2. How breakable walls are generated? 7

2.3. How enemies are moved? 8

2.4. How bombs explode? 8

3. Documentation 9

3.1. Directions enum 9

3.2. MoveResult enum 9

3.3. Player struct 9

3.4. Enemy struct 9

3.5. Bomb struct 10

3.6. GameInfo struct 10

3.7. GameField struct 10

Introduction. Short description of task and documentation

Original task:

Pomocou knižnice ncurses vytvorte ľubovoľný program (hru, prezentáciu alebo iný umelecký počin), pričom výsledný projekt musí spĺňať nasledujúce podmienky:

- Projekt musí obsahovať 2D svet.
- Dohromady musí projekt zahŕňať aspoň 3 z nasledujúcich výziev:
    - Práca s farbami
    - Ovládanie cez klávesnicu (bez nutnosti potvrdenia Entrom)
    - Viac úrovní (levelov)
    - Práca s časomierou resp. práca v čase (s časom sa program mení)
    - Práca s argumentami príkazového riadku
    - Práca so súbormi
- Projekt musí byť zložitejší ako ukážkové príklady a jeho úroveň musí byť dostatočná.

As base game for my Problem Set 6 project I choose an old-style popular game – Bomberman.

Functionalities that I implemented in gameplay as for now:

- Game consists of 5 levels.
- Map of each level, player's and enemies' positions are generated randomly.
- Tick-based gameplay. One tick is counted on players move, or specified time, if player doesn't move or press any key.
- Placing of bombs, and their explosion after specified number of ticks, which will affect predefined number of tiles.
- Bomb's explosion will kill player, enemies and brake walls in its range
- Moving on next level through door, which is hidden under breakable wall and is not visible, until at least one enemy is killed on same level.
- Counting of game's score.
- Walls and borders are unpassable for enemies and player.
- Semi-random moving pattern of enemies, based on "cost" (in my implementation: priority) of tiles surrounding him in directions of moving.

Features planned for development in future are listed in Conclusion section.

1. How to compile and run the program

1.1. Debian-based systems

Run in your terminal:

sudo apt update

Then, install build tool, compiler and ncurses library for C:

sudo apt-get install gcc make libncurses5-dev libncurses5

Download or clone project from repository. Then step into directory, which contains Makefile and src directory (bomberman in this tutorial):

cd bomberman

Run project build command:

make

You can run game now:

./bomberman

To clean binary and build files, run:

make clean

1.2. Arch-based systems

Update packages info:

sudo pacman -Syu

Install required tools:

sudo pacman -S make gcc ncurses

Download or clone project from repository. Then step into directory, which contains Makefile and src directory (bomberman in this tutorial):

cd bomberman

Run project build command:

make

You can run game now:

./bomberman

To clean binary and build files, run:

make clean

2. Solutions I used to write the game.

Here I will show how I implemented some parts of the game.

2.1. Game in general

First of all, my game is initializing ncurses parts: screen colours, background. Game has three main loops. Now I will go from inner loop to outer loop. First inner loop is responsible for handling one player's move and triggering one game "tick". Middle loop generates one level of the game and all information related to it. Outer loop controls each game: player starts game, plays, in case of loose or win, he will be prompted if he wants to restart game or leave. If player decides to restart game, he goes on another loop iteration. In outer loop GameInfo variable is created, containing info about current level and score on all levels. In the middle loop are generated main parts of game: field, enemies, player, breakable walls. On the start of inner loop, all instances are displayed on screen. User input blocks loop for small amount of time. If user presses any key, game ticks at once (after handling user input). Game tick includes enemies moving and bombs explosion. After tick and the end of the inner loop, player's state is checked, if he is dead, game lost. After loose user prompted to leave or to start new game. If for the end of inner loop player has found a door, new iteration in middle loop is started and all instances are recreated.

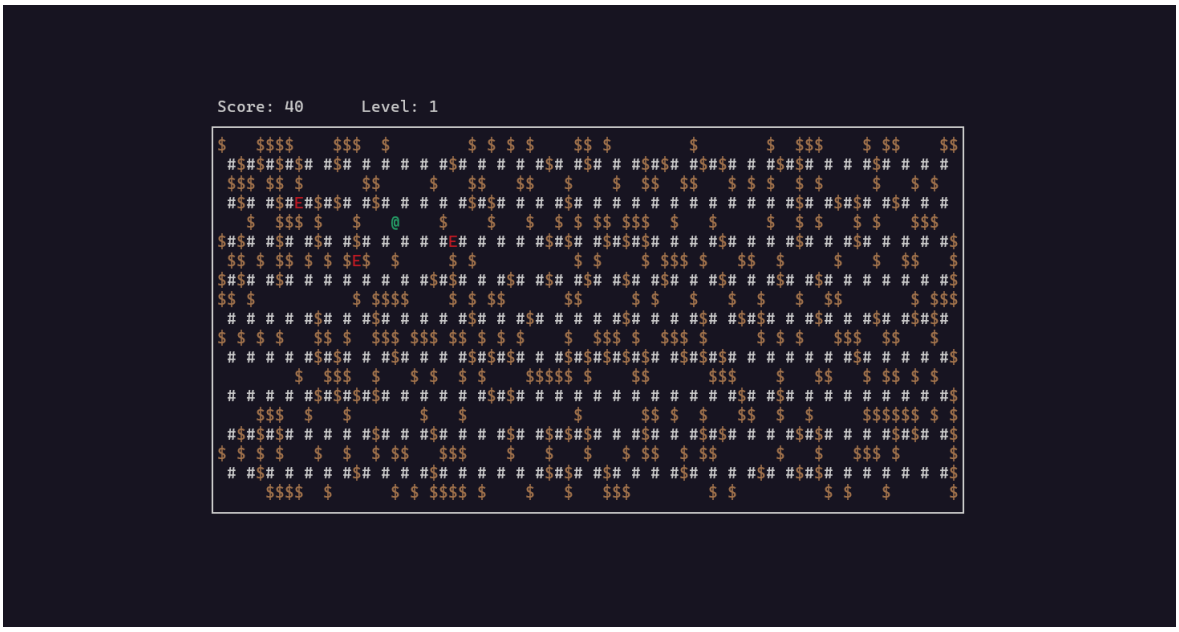

Fig. 1 View of the game.

2.2. How breakable walls are generated?

First of all, with new level start map grid, or solid walls are generated. Then, on each line walls generated on empty places. I used for it random numbers generator.

2.3. How enemies are moved?

Each enemy before move firstly check each tile around. If tile is passable (does not contain wall or another enemy), then, this direction is saved to array. If array size is 0, then there are not any passable tiles and enemy stays on its place. Then, priority of each passable tile is calculated: priority of tile with player is 2, empty 1 and bomb 0. So, if enemy stays beside player, it will go in his direction. Also, enemy will try to escape bomb by moving in any another direction except bomb tile, if such tiles exist. Enemies are moved each tick.

2.4. How bombs explode?

Each new bomb is created with 5 ticks value until explosion. Each tick this value is decreased by 1. Also, every tick program checks every bomb, if “time” until explosion is 0, then bomb explodes. Firstly, tile with bomb is checked. If there stays player, game will end, enemy will be killed. Then, each direction is checked and similar conditions are applied. Additionally, bomb will brake breakable walls.

3. Documentation

I will describe here all enumerations and structs, that were used in program and their fields.

3.1. Directions enum

Contains four directions in which enemy can be moved. Directions named by sides of the world.

Values list:

- NORTH
- EAST
- SOUTH
- WEST
- FAILED

3.2. MoveResult enum

Contains all possible outcomes of player's move. Values list:

- LOST
- FAILURE
- STAND
- SUCCESS
- WIN

3.3. Player struct

Represents player entity and in-game information. New entity created every on every new level.

Fields:

- **int** x – x coordinate on screen
- **int** y – y coordinate on screen
- **bool** is_dead – is player died from any condition. **False** by default
- **bool** is_door_found – is player found a door and can move forward. **False** by default

3.4. Enemy struct

Structure which contains information about enemy. Fields:

- **int** x – x coordinate on screen
- **int** y – y coordinate on screen

3.5. Bomb struct

Structure which implements doubly linked list of all bombs present on screen. Fields:

- **int** x – x coordinate on screen
- **int** y – y coordinate on screen
- **int** till_explode – number of ticks till bomb explosion. **5** by default
- **struct Bomb *prev** – pointer on previous node. Has **null** value if first node in list
- **struct Bomb *next** – pointer on next node. Has **null** value if last node in list

3.6. GameInfo struct

Main game info storage. Recreated on new game start. Fields:

- **int** score – game score. **0** by default
- **int** level – game level. **1** by default
- **bool** is_lost – is player lost game. **False** by default

3.7. GameField struct

Contains info about current level. Fields:

- **WINDOW *window** – current level window
- **int** height – height of window
- **int** width – width of window
- **int** bomb_range – bomb range. **2** by default
- **int** local_score – local level score. **0** by default

Conclusion

I found this game an excellent way to learn new things and improve my C skills. Game contains many features from original version, but there are still things to do.

What I would like to improve:

- Game states management need to be changed. When making changes to game's state, program should monitor entities by indexes stored in program, not chars on this indexes that are displayed on screen
- I want to improve algorithm which enemies us for moving. As example of such algorithm I would like to implement A* pathfinding algorithm, which finds the best path from point to point using heuristics.
- Different types of enemies will make game more dynamic.
- On-level bonuses will be generated